LAPORAN PRATIKUM

STRUKTUR DATA

ABSTRACT DATA TYPE JAM

disusun Oleh:

Annisa Layli Ramadhani

2511532024

Dosen Pengampu: Wahyudi. Dr.. S.T.M.T

Asisten Pratikum: Muhammad Galid Avero

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2025

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salaj satu bentuk pemenuhan tugas praktikum serta untuk memahami penerapan konsep Abstract Data Type (ADT) dalam bahasa java.

Dalam laporan ini dibahas implementasi kelas jam yang mencakup berbagai operasi seperti validasi, konstruktor, selektor, mutator, serta operasi konversi dan aritmatika waktu. Melalui praktikum ini, diharapkan penulis dapat lebih memahami konsep pemrograman berorientasi objek secara lebih mendalam.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa yang akan datang. Semoga laporan ini dapat bermanfaat bagi pembaca.

Padang, 31 Maret 2026

Penulis

DAFTAR ISI

KATA PENGANTAR ..... **ii**
DAFTAR ISI ..... **iii**
BAB I PENDAHULUAN **4**
1.1 Latar Belakang **4**
1.2 Tujuan **4**
1.3 Manfaat **5**
BAB II PEMBAHASAN **6**
2.1 Kode Program Pekan 1 ..... **6**
2.1.1 Jam_2511532024 ..... **6**
2.1.2 JamDriver_2511532024 ..... **9**
2.1.3 JamDriver2_2511532024 ..... **10**
BAB III PENUTUP **13**
3.1 Kesimpulan **13**
DAFTAR PUSTAKA **14**

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menuntut pemahaman yang baik terhadap konsep dasar pemrograman, khususnya dalam pengelolaan data secara terstruktur. Salah satu konsep penting dalam ilmu komputer adalah ADT, yang memungkinkan suatu data dipresentasikan bersama dengan operasi-operasi yang dapat dilakukan terhadap data tersebut.

Dalam pemrograman berorientasi objek, ADT dapat diimplementasikan melalui pembuatan kelas yang mencerminkan suatu objek nyata. Salah satu contoh yang sederhana namun penting adalah representasi waktu dalam bentuk jam, menit, dan detik. Pengelolaan data waktu waktu sering digunakan dalam berbagai aplikasi, sehingga diperlukan struktur yang jelas dan operasi yang tepat untuk memanipulasinya.

Oleh karena itu, pada praktikum ini dilakukan implementasi ADT jam menggunakan bahasa java. Melalui pembuatan kelas ini, diharapkan mahasiswa dapat memahami bagaimana cara mendefinisikan atribut, membuat konstruktor, serta mengimplementasikan berbagai operasi seperti validasi, konversi, dan perhitungan waktu secara sistematis dan terstruktur.

1.2 Tujuan

Tujuan dari pelakasanaan praktikum antara lain sebagai berikut:

1. Memahami konsep ADT dalam pemrograman.
2. Menerapkan konsep ADT ke dalam bahasa java melalui pembuatan kelas jam.
3. Memahami penggunaan konstruktor, selektor, mutator, serta operasi-operasi pada objek.

4. Melatih kemampuan dalam mengelola dan memanipulasi data waktu secara terstruktur.

1.3 Manfaat

Manfaat dari pelaksanaan praktikum antara lain sebagai berikut:

1. Meningkatkan pemahaman tentang pemrogram berorientasi objek.
2. Membantu dalam memahami cara kerja representasi data waktu dalam program.
3. Menjadi dasar dalam pengembangan program yang lebih kompleks.
4. Melatih kemampuan berpikir logis dan sistematis dalam pemrograman.

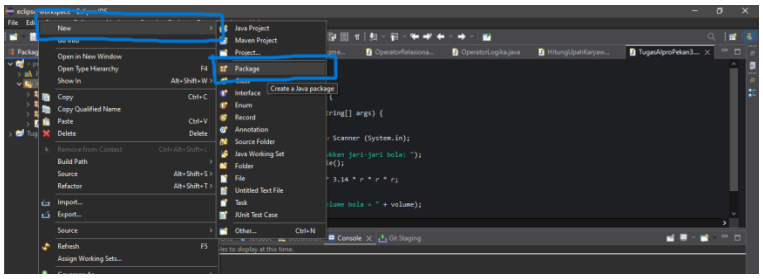
BAB II

PEMBAHASAN

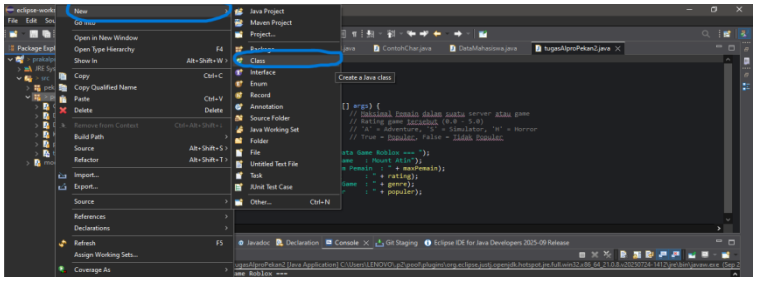
2.1 Kode Program Pekan 1

2.1.1 Jam_2511532024

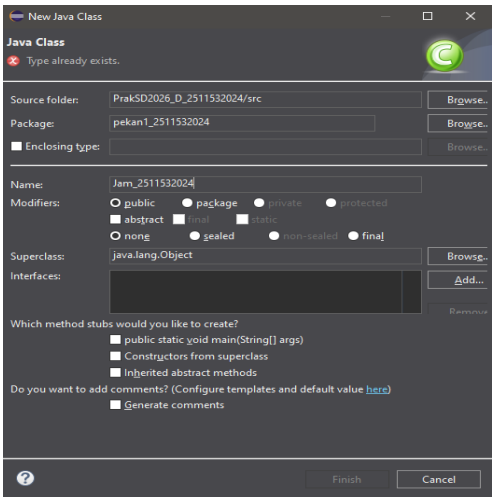
1. Buka aplikasi eclipse, lalu buat package baru pada bagian src klik kanan, lalu pilih “New” dan klik tulisan package.



2. Kemudian buat nama packagenya tanpa spasi, huruf kapital, karakter khusus. Beri nama “pekan1_2511532024”. Lalu klik finish.
3. Ketika package sudah jadi. Klik pada bagian “pekan1_2511532024”. Lalu klik “New” dan pilih bagian “Class” untuk memulai membuat program.



4. Buat nama Class program yang akan dibuat pada bagian nama tanpa spasi dan menggunakan huruf kapital pada awal kata, lalu klik bagian “public static void main(string[]args)”. Kemudian klik “finish”.



5. Masukkan syntax berikut.

```
1. public final class Jam_2511532024 {
2.             private int hh; // 0..23
3.             private int mm; // 0..59
4.             private int ss; // 0..59
5.
6.             // --- Validator ---
7.             public static boolean isValid(int h,
   int m, int s) {
8.                     return (0 <= h && h <= 23) && (0
   <= m && m <= 59) && (0 <= s && s <= 59);
9.             }
10.
11.            // --- Konstruktor/Kreator ---
12.            public Jam_2511532024(int h, int m, int
   s) {
13.                    this.hh = h; this.mm = m;
   this.ss = s;
14.            }
15.
16.            // --- Selektor ---
17.            public int getHour() {
18.                    return hh;
19.            }
20.            public int getMinute() {
21.                    return mm;
22.            }
23.            public int getSecond() {
24.                    return ss;
25.            }
26.
27.            // --- Mutator ---
28.            public void setHour(int h) {
29.                    this.hh = h;
30.            }
31.            public void setMinute(int m) {
32.                    this.mm = m;
33.            }
34.            public void setSecond(int s) {
35.                    this.ss = s;
```

```java
        }

        // --- Konversi ---
        public int toSeconds( ) { return hh * 3600 + mm * 60 + ss;}
        public static Jam_2511532024 fromSeconds(int total) {
            if (total < 0) throw new IllegalArgumentException("detik negatif");
            total %= 24 * 3600; // wrap 24 jam
            int h = total / 3600;
            total %= 3600;
            int m = total / 60;
            int s = total % 60;
            return new Jam_2511532024(h,m,s);
        }

        // --- Relasional ---
        public int compareTo(Jam_2511532024 other) {
            return Integer.compare(this.toSeconds(), other.toSeconds());
        }
        public boolean equals(Object o) {
            if (!(o instanceof Jam_2511532024 j)) return false;
            return hh == j.hh && mm == j.mm && ss == j.ss;
        }
        public int hashCode() { return java.util.Objects.hash(hh, mm, ss);}

        // --- Aritmatika ---
        public Jam_2511532024 plus(Jam_2511532024 other) { return fromSeconds(this.toSeconds() + other.toSeconds()); }
        public Jam_2511532024 minus(Jam_2511532024 other) { return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24*3600)); }
        public Jam_2511532024 nextSecond() { return fromSeconds(this.toSeconds() + 1); }
        public Jam_2511532024 nextNSeconds(int n) { return fromSeconds(this.toSeconds() + Math.max(0, n)); }
        public Jam_2511532024 prevSecond() { return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24*3600)); }
        public Jam_2511532024 prevNSeconds(int n) { return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24*3600)); }

```

```
68.             // --- Durasi (detik) -- bisa neagtif
    jika this > other sesuai spesifikasi durasi(JAW,
    JAkh)
69.             public static int
    durasiDetik(Jam_2511532024 jaw, Jam_2511532024
    jakh) { return jakh.toSeconds() -
    jaw.toSeconds(); }
70.             // overriding ke string
71.             public String toString() { return
    String.format("%02d:%02d:%02d", hh, mm, ss); }
72.     }
```

2.1.2 JamDriver_2511532024

1. Ulangi pembuatan Class seperti program sebelumnya dan beri nama Class tersebut “JamDriver_2511532024”. Lalu masukkan syntax seperti berikut.

```
1.  public class JamDriver_2511532024 {
2.
3.      public static void main(String[] args) {
4.              Jam_2511532024 a = new Jam_2511532024
    (23, 59, 50);
5.              Jam_2511532024 b = new Jam_2511532024
    (0, 0, 15);
6.              System.out.println("a              = "
    + a);
7.              System.out.println("b              = "
    + b);
8.              System.out.println("a+b            = "
    + a.plus(b));
9.              System.out.println("next 20s       = "
    + a.nextNSeconds(20));
10.             System.out.println("durasi(a,b)    = "
    + Jam_2511532024.durasiDetik(a, b));
11.             System.out.println("a.compareTo(b) = "
    + a.compareTo(b));
12.     }
13. }
```

2. Klik “run” untuk melihat hasil syntax yang sudah dibuat.

```
<terminated> JamDriver_2511532024 [Java A
a               = 23:59:50
b               = 00:00:15
a+b             = 00:00:05
next 20s        = 00:00:10
durasi(a,b)     = -86375
a.compareTo(b)  = 1
```

2.1.3 JamDriver2_2511532024

1. Ulangi pembuatan Class seperti program sebelumnya dan beri nama Class tersebut “Nilai”. Lalu masukkan syntax seperti berikut.

```
import java.util.Scanner;

public class JamDriver2_2511532024 {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("=== Program Driver Objek JAM ===");
        // 1. Input Jam pertama
        System.out.println("\n--- Input Jam 1 ---");
        Jam_2511532024 j1 = buatJamDariInput(input);

        // 2. Input Jam Kedua
        System.out.println("\n--- Input Jam 2 ---");
        Jam_2511532024 j2 = buatJamDariInput(input);

        // 3. Menampilkan Data
        System.out.println("\n--- Hasil Operasi ---");
        System.out.println("Jam 1 (String)          : " + j1.toString());
        System.out.println("Jam 2 (String)          : " + j2.toString());
        System.out.println("Jam 1 dalam detik       : " + j1.toSeconds());
        System.out.println("Jam 2 dalam detik       : " + j2.toSeconds());

        // 4. Operasi Relasional (Perbandingan)
        int perbandingan = j1.compareTo(j2);
        if (perbandingan > 0) {
            System.out.println("Status                  : Jam 1 lebih lambat (setelah) Jam 2");
        }
        else if (perbandingan < 0) {
            System.out.println("Status                  : Jam 1 lebih awal (sebelum) Jam 2");
        }
        else {
            System.out.println("Status                  : Jam 1 dan Jam 2 sama persis");
        }

```

```java
            // 5. Operasi Aritmatika
            System.out.println("Durasi (J1 ke J2)         : " + Jam_2511532024.durasiDetik(j1, j2) + " detik");

            Jam_2511532024 jNext = j1.nextSecond();
            System.out.println("Jam 1 Detik Berikutnya    : " + jNext);

            Jam_2511532024 jPrev = j1.prevSecond();
            System.out.println("Jam 1 Detik Sebelumnya    : " + jPrev);

            // 6. Operasi Penjumlahan Jam
           Jam_2511532024 jHasilPlus = j1.plus(j2);
            System.out.println("Hasil J1 + J2             : " + jHasilPlus);

            input.close();
           System.out.println("\nProgram  Selesai");

     }
            /**
             * Prosedur pembantu untuk melakukan input dan validasi secara berulang
             * sampai user memasukkan angka yang benar (0..23, 0..59)
             */

     private static Jam_2511532024 buatJamDariInput(Scanner sc) {
            int h, m, s;
            while (true) {
                   System.out.print("Masukkan Jam (0-23)       : ");
                   h = sc.nextInt();
                   System.out.print("Masukkan Menit (0-59)     : ");
                   m = sc.nextInt();
                   System.out.print("Masukkan Detik (0-59)     : ");
                   s = sc.nextInt();

                   // Menampilkan method static isValid dari keals Jam
                   if (Jam_2511532024.isValid(h, m, s)) {
                          return new Jam_2511532024(h, m, s);
                   } else {
                          System.out.println("[Error] Input tidak valid! Silakan ulangi. \n");
                   }
            }
     }
}
```

2. Klik “run” untuk melihat hasil syntax yang sudah dibuat.

```
=== Program Driver Objek JAM ===

--- Input Jam 1 ---
Masukkan Jam (0-23)      : 09
Masukkan Menit (0-59)    : 10
Masukkan Detik (0-59)    : 06

--- Input Jam 2 ---
Masukkan Jam (0-23)      : 11
Masukkan Menit (0-59)    : 04
Masukkan Detik (0-59)    : 00

--- Hasil Operasi ---
Jam 1 (String)           : 09:10:06
Jam 2 (String)           : 11:04:00
Jam 1 dalam detik        : 33006
Jam 2 dalam detik        : 39840
Status                   : Jam 1 lebih awal (sebelum) Jam 2
Durasi (J1 ke J2)        : 6834 detik
Jam 1 Detik Berikutnya   : 09:10:07
Jam 1 Detik Sebelumnya   : 09:10:05
Hasil J1 + J2            : 20:14:06

Program Selesai
```

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa konsep ADT dapat diimplementasikan dengan baik menggunakan bahasa pemrograman Java melalui pembuatan kelas Jam. Kelas ini mampu merepresentasikan data waktu secara terstruktur dengan atribut jam, menit, dan detik serta dilengkapi dengan berbagai operasi seperti validasi, konversi, perbandingan, dan aritmatika waktu.

Penggunaan metode konversi ke detik menjadi inti dari proses perhitungan, sehingga operasi yang dilakukan menjadi lebih sederhana dan efisien. Selain itu, penerapan konsep enkapsulasi melalui penggunaan atribut private serta method getter dan setter membantu menjaga keamanan dan konsisten data.

Dengan demikian, praktikum ini memberikan pemahaman yang lebih baik mengenai penerapan ADT dan pemrograman berorientasi objek, serta melatih kemampuan dalam merancang program yang sistematis dan terstruktur.

DAFTAR PUSTAKA

[1] Oracle, “Java Platform, Standard Edition Documentation,” [Online]. Available: https://docs.oracle.com/javase/8/docs/. [Diakses: 31 Maret 2026].